

Time Complexity

Content notes, classwork and homework with worked C++ solutions.

Lesson 1: Time Complexity I

Quick recap: What is Time Complexity? Big-O notation intro • Linear time $O(N)$ vs Constant time $O(1)$ • Best, worst, and average cases • Ignoring constants

Theory & Nuances

What is Time Complexity? Big-O notation intro

Time complexity does not measure seconds - seconds change with the laptop, the compiler and what else is running. Instead we count how many basic steps (a comparison, an addition, an assignment) a program performs and ask how that count grows as the input size N grows. Big-O notation, written $O(f(N))$, describes an upper bound on that growth: $O(N)$ means 'roughly proportional to N steps in the worst case'.

Linear time $O(N)$ vs Constant time $O(1)$

An operation is $O(1)$ when it takes the same number of steps no matter how big N is - reading a variable, a single formula like $n * (n + 1) / 2$, or accessing `arr[i]`. A loop that touches each of the N items once is $O(N)$: double N and the work doubles. Summing $1..N$ with a loop is $O(N)$; summing it with the formula is $O(1)$.

Best, worst, and average cases

The same program can do different amounts of work for different inputs of the same size. When searching N numbers for a target, the best case is finding it at the first position (1 comparison), the worst case is finding it last or not at all (N comparisons), and the average case is about $N/2$. Big-O is normally quoted for the worst case because it is a guarantee. Omega (Ω) gives a lower bound (best case) and Theta (Θ) is used when the upper and lower bounds match.

Ignoring constants

Big-O keeps only the fastest-growing term and drops constant factors: $3N + 5$ becomes $O(N)$, $N/2$ becomes $O(N)$, and 100 stays $O(1)$. The reason is that for very large N the leading term dominates everything else, and constant factors depend on the machine anyway. Two loops one after the other ($N + N = 2N$) are still $O(N)$.

Key Points / Common Mistakes

- Time complexity counts steps as a function of N , not seconds on a clock.
- Two separate loops of N each is $2N = O(N)$, not $O(N^2)$ - only nested loops multiply.
- A while loop that halves N runs about $\log_2(N)$ times, which grows much slower than N .
- Quote Big-O for the worst case unless the question asks for best or average.

Classwork Problems

1. Calculate TC of a standard for loop

```
#include <iostream>
using namespace std;

int main() {
    int n = 10;
    int count = 0;

    for (int i = 1; i <= n; i++) {
        count++;
    }

    cout << "N = " << n << endl;
    cout << "Loop body ran " << count << " times" << endl;
    cout << "Steps grow in step with N, so TC = O(N)" << endl;
    return 0;
}
```

Output

```
N = 10
Loop body ran 10 times
Steps grow in step with N, so TC = O(N)
```

2. Calculate TC of a while loop halving N

```
#include <iostream>
using namespace std;

int main() {
    int n = 64;
    int temp = n;
    int count = 0;

    while (temp > 1) {
        temp = temp / 2;
        count++;
    }

    cout << "N = " << n << endl;
    cout << "Halved " << count << " times to reach 1" << endl;
    cout << "64 = 2^6, so the loop runs log2(N) times: TC = O(log N)" << endl;
    return 0;
}
```

Output

```
N = 64
Halved 6 times to reach 1
64 = 2^6, so the loop runs log2(N) times: TC = O(log N)
```

3. Differentiate $O(1)$ ops from $O(N)$ ops

```
#include <iostream>
using namespace std;

int main() {
    int n = 8;

    //  $O(N)$ : add 1 + 2 + ... + n one number at a time
    int loopSum = 0, loopSteps = 0;
    for (int i = 1; i <= n; i++) {
        loopSum += i;
        loopSteps++;
    }

    //  $O(1)$ : one formula, no matter how big n is
    int formulaSum = n * (n + 1) / 2;
    int formulaSteps = 1;

    cout << "Loop sum = " << loopSum << " in " << loopSteps << " steps ->  $O(N)$ " << endl;
    cout << "Formula sum = " << formulaSum << " in " << formulaSteps << " step ->  $O(1)$ " << endl;
    return 0;
}
```

Output

```
Loop sum = 36 in 8 steps ->  $O(N)$ 
Formula sum = 36 in 1 step ->  $O(1)$ 
```

Homework Problems

1. Best vs worst case of linear search (Ω , O and Θ)

```
#include <iostream>
using namespace std;

int main() {
    int arr[6] = {7, 3, 9, 4, 1, 8};
    int n = 6;

    // Best case: the target is the very first element
    int target = 7, comparisons = 0;
    for (int i = 0; i < n; i++) {
        comparisons++;
        if (arr[i] == target) break;
    }
    cout << "Searching " << target << ": " << comparisons << " comparison(s) -> best case,  $\Omega(1)$ " << endl;

    // Worst case: the target is the last element
    target = 8;
    comparisons = 0;
    for (int i = 0; i < n; i++) {
        comparisons++;
        if (arr[i] == target) break;
    }
    cout << "Searching " << target << ": " << comparisons << " comparison(s) -> worst case,  $O(N)$ " << endl;

    cout << "Bounds differ, so linear search has no single  $\Theta$  for all inputs" << endl;
    return 0;
}
```

Output

```
Searching 7: 1 comparison(s) -> best case,  $\Omega(1)$ 
Searching 8: 6 comparison(s) -> worst case,  $O(N)$ 
Bounds differ, so linear search has no single  $\Theta$  for all inputs
```

2. TC of finding max element in array

```
#include <iostream>
using namespace std;

int main() {
    int arr[7] = {12, 45, 7, 89, 23, 56, 34};
    int n = 7;
    int maxVal = arr[0];
    int comparisons = 0;

    for (int i = 1; i < n; i++) {
        comparisons++;
        if (arr[i] > maxVal) {
            maxVal = arr[i];
        }
    }

    cout << "Maximum element: " << maxVal << endl;
    cout << "Comparisons made: " << comparisons << " (N - 1)" << endl;
    cout << "Every element must be checked, so TC = Theta(N)" << endl;
    return 0;
}
```

Output

```
Maximum element: 89
Comparisons made: 6 (N - 1)
Every element must be checked, so TC = Theta(N)
```

Lesson 2: Time Complexity II

Quick recap: Analyzing nested loops $O(N^2)$ • Dependent vs independent nested loops • Introduction to logarithmic time $O(\log N)$

Theory & Nuances

Analyzing nested loops $O(N^2)$

When one loop sits inside another, the inner loop runs completely for every single iteration of the outer loop. If the outer loop runs N times and the inner loop runs N times, the body runs $N \times N = N^2$ times. That is why printing an $N \times N$ square pattern is $O(N^2)$: doubling N makes the work four times bigger. Each extra level of nesting multiplies by another N , so three nested loops give $O(N^3)$.

Dependent vs independent nested loops

In an independent nested loop the inner bound does not use the outer variable (j from 1 to N), so the total is simply $N \times N$. In a dependent loop the inner bound uses the outer variable (j from 1 to i): it runs 1, then 2, then 3 ... then N times, for a total of $1 + 2 + \dots + N = N(N + 1)/2$. Expanding gives $N^2/2 + N/2$, and after dropping constants and lower terms that is still $O(N^2)$ - roughly half the work, but the same growth.

Introduction to logarithmic time $O(\log N)$

If the loop variable is multiplied ($i *= 2$) or divided ($n /= 2$) on each pass instead of increased by a fixed amount, it reaches N in only about $\log_2(N)$ steps. For $N = 1,000,000$ that is about 20 iterations instead of a million. Note that $i += 2$ is still linear ($N/2$ steps = $O(N)$) - only multiplying or dividing gives the logarithm.

Key Points / Common Mistakes

- Nested loops multiply, sequential loops add: N then N is $O(N)$, N inside N is $O(N^2)$.
- Dependent loops (j up to i) do about half the work but are still $O(N^2)$.
- $i += 2$ is $O(N)$; $i *= 2$ is $O(\log N)$. Do not confuse the two.
- Starting $i *= 2$ at 0 is an endless loop - always start at 1.

Classwork Problems

1. TC of nested loop (1 to N, 1 to N)

```
#include <iostream>
using namespace std;

int main() {
    int n = 4;
    int count = 0;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            count++;
        }
    }

    cout << "N = " << n << endl;
    cout << "Inner body ran " << count << " times" << endl;
    cout << "N * N = " << n * n << ", so TC = O(N^2)" << endl;
    return 0;
}
```

Output

```
N = 4
Inner body ran 16 times
N * N = 16, so TC = O(N^2)
```

2. TC of dependent loop (i from 1 to N, j from 1 to i)

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    int count = 0;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            count++;
        }
    }

    cout << "N = " << n << endl;
    cout << "Inner body ran " << count << " times" << endl;
    cout << "1 + 2 + ... + N = N(N+1)/2 = " << n * (n + 1) / 2 << endl;
    cout << "Dropping constants, TC = O(N^2)" << endl;
    return 0;
}
```

Output

```
N = 5
Inner body ran 15 times
1 + 2 + ... + N = N(N+1)/2 = 15
Dropping constants, TC = O(N^2)
```

3. TC of $i *= 2$ loop ($\log N$)

```
#include <iostream>
using namespace std;

int main() {
    int n = 100;
    int count = 0;

    for (int i = 1; i <= n; i *= 2) {
        cout << i << " ";
        count++;
    }
    cout << endl;

    cout << "N = " << n << ", loop ran " << count << " times" << endl;
    cout << "i doubles each time, so TC =  $O(\log N)$ " << endl;
    return 0;
}
```

Output

```
1 2 4 8 16 32 64
N = 100, loop ran 7 times
i doubles each time, so TC =  $O(\log N)$ 
```

Homework Problems

1. TC of three nested loops ($O(N^3)$)

```
#include <iostream>
using namespace std;

int main() {
    int n = 3;
    int count = 0;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            for (int k = 1; k <= n; k++) {
                count++;
            }
        }
    }

    cout << "N = " << n << endl;
    cout << "Innermost body ran " << count << " times" << endl;
    cout << "N * N * N = " << n * n * n << ", so TC =  $O(N^3)$ " << endl;
    return 0;
}
```

Output

```
N = 3
Innermost body ran 27 times
N * N * N = 27, so TC =  $O(N^3)$ 
```

2. TC of loop jumping by $i += 2$ vs $i *= 2$

```
#include <iostream>
using namespace std;

int main() {
    int n = 32;

    int addSteps = 0;
    for (int i = 1; i <= n; i += 2) {
        addSteps++;
    }

    int mulSteps = 0;
    for (int i = 1; i <= n; i *= 2) {
        mulSteps++;
    }

    cout << "N = " << n << endl;
    cout << "i += 2 ran " << addSteps << " times -> N/2 -> O(N)" << endl;
    cout << "i *= 2 ran " << mulSteps << " times -> log2(N) + 1 -> O(log N)" << endl;
    return 0;
}
```

Output

```
N = 32
i += 2 ran 16 times -> N/2 -> O(N)
i *= 2 ran 6 times -> log2(N) + 1 -> O(log N)
```

Lesson 3: Time & Space Complexity

Quick recap: Space complexity: auxiliary space vs input space • Recursion stack space intro • Trade-offs between Time and Space

Theory & Nuances

Space complexity: auxiliary space vs input space

Space complexity measures how much memory a program needs as N grows. Input space is the memory taken by the input itself (an array of N numbers is N ints). Auxiliary space is the extra memory the algorithm creates on top of that - counters, temporary arrays, copies. When people ask 'what is the space complexity of this algorithm' they usually mean auxiliary space: a few int variables is $O(1)$, a new array of size N is $O(N)$, an $N \times N$ matrix is $O(N^2)$.

Recursion stack space intro

Every time a function calls itself, the computer keeps the unfinished call on a stack (like a pile of plates) until the inner call returns. A recursive function that goes N levels deep keeps N calls alive at the same time, so it uses $O(N)$ stack space even if it creates no arrays. This is why very deep recursion can crash with a 'stack overflow'.

Trade-offs between Time and Space

Often you can make a program faster by remembering extra information. A prefix-sum array costs $O(N)$ extra memory but lets you answer 'sum from index l to r ' in $O(1)$ instead of $O(N)$. A 'seen' array lets you spot duplicates in one pass. The reverse also holds: if memory is tight you can recompute instead of store. Choosing between them is the time-space trade-off.

Key Points / Common Mistakes

- A handful of variables is $O(1)$ space no matter how large their values are.
- Do not forget the recursion stack - recursion N levels deep is $O(N)$ space.
- An $R \times C$ matrix uses $R * C$ cells; for $N \times N$ that is $O(N^2)$.
- Swapping in place (two pointers) avoids an $O(N)$ copy of the array.

Classwork Problems

1. SC of creating an array of size N

```
#include <iostream>
using namespace std;

int main() {
    const int N = 5;
    int arr[N];

    for (int i = 0; i < N; i++) {
        arr[i] = (i + 1) * 10;
    }

    for (int i = 0; i < N; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    cout << "Array holds " << N << " ints = " << N * sizeof(int) << " bytes" << endl;
    cout << "Memory grows with N, so SC = O(N)" << endl;
    return 0;
}
```

Output

```
10 20 30 40 50
Array holds 5 ints = 20 bytes
Memory grows with N, so SC = O(N)
```

2. SC of basic recursion

```
#include <iostream>
using namespace std;

int depth = 0, maxDepth = 0;

int sumTo(int n) {
    depth++;
    if (depth > maxDepth) maxDepth = depth;

    int result;
    if (n == 0) result = 0;
    else result = n + sumTo(n - 1);

    depth--;
    return result;
}

int main() {
    int n = 4;
    int total = sumTo(n);

    cout << "Sum 1 to " << n << " = " << total << endl;
    cout << "Deepest point: " << maxDepth << " calls waiting on the stack" << endl;
    cout << "Stack grows with N, so SC = O(N)" << endl;
    return 0;
}
```

Output

```
Sum 1 to 4 = 10
Deepest point: 5 calls waiting on the stack
Stack grows with N, so SC = O(N)
```

3. Time-Space trade-off with prefix sums

```
#include <iostream>
using namespace std;

int main() {
    int arr[6] = {3, 1, 4, 1, 5, 9};
    int n = 6;
    int l = 1, r = 4;

    // Way 1: no extra memory, loop over the range every time -> O(N) per query
    int sum1 = 0, steps1 = 0;
    for (int i = l; i <= r; i++) {
        sum1 += arr[i];
        steps1++;
    }

    // Way 2: build prefix[] once (O(N) extra space), then each query is O(1)
    int prefix[7];
    prefix[0] = 0;
    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i] + arr[i];
    }
    int sum2 = prefix[r + 1] - prefix[l];

    cout << "Sum of arr[" << l << ".." << r << "]" << endl;
    cout << "Loop: " << sum1 << " in " << steps1 << " steps, O(1) extra space" << endl;
    cout << "Prefix: " << sum2 << " in 1 step, O(N) extra space" << endl;
    return 0;
}
```

Output

```
Sum of arr[1..4]
Loop: 11 in 4 steps, O(1) extra space
Prefix: 11 in 1 step, O(N) extra space
```

Homework Problems

1. Calculate SC of nested arrays (matrices)

```
#include <iostream>
using namespace std;

int main() {
    const int R = 3, C = 4;
    int mat[R][C];
    int cells = 0;

    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            mat[i][j] = i * C + j;
            cells++;
        }
    }

    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            cout << mat[i][j] << "\t";
        }
        cout << endl;
    }

    cout << "Cells used: " << cells << " = R * C" << endl;
    cout << "For an N x N matrix that is N^2, so SC = O(N^2)" << endl;
    return 0;
}
```

Output

```
0■1■2■3■
4■5■6■7■
8■9■10■11■
Cells used: 12 = R * C
For an N x N matrix that is N^2, so SC = O(N^2)
```

2. O(1) space vs O(N) space: reverse an array

```
#include <iostream>
using namespace std;

int main() {
    int a[5] = {1, 2, 3, 4, 5};
    int n = 5;

    // O(N) space: copy into a new array backwards
    int b[5];
    for (int i = 0; i < n; i++) {
        b[i] = a[n - 1 - i];
    }
    cout << "Copy reversed: ";
    for (int i = 0; i < n; i++) cout << b[i] << " ";
    cout << endl;

    // O(1) space: swap the two ends and move inwards
    int left = 0, right = n - 1;
    while (left < right) {
        int temp = a[left];
        a[left] = a[right];
        a[right] = temp;
        left++;
        right--;
    }
    cout << "In-place reversed: ";
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
    return 0;
}
```

Output

```
Copy reversed:    5 4 3 2 1
In-place reversed: 5 4 3 2 1
```